QUESTION 1

Title: Analyze the Effect of Recursive Stack Depth in Merge Sort on Systems with Limited Memory

Aim:

To analyze the impact of recursive stack depth in Merge Sort by measuring execution time and recursion depth, and to study how recursion affects memory utilization in systems with limited memory.

Problem Understanding:

Merge Sort is a divide-and-conquer algorithm that recursively divides an array into smaller subarrays until each subarray contains a single element. Each recursive call occupies space in the system call stack.

As input size increases, the recursion depth increases approximately as $\log_2(n)$. This experiment evaluates stack usage and execution time to understand the memory overhead caused by recursion and discusses optimization techniques to reduce stack consumption.

Algorithm:

1. Generate arrays of different sizes.
2. Implement Merge Sort recursively.
3. Track maximum recursion depth.
4. Measure execution time.
5. Display recursion depth and execution time.
6. Analyze stack utilization.

Pseudocode:

```
MERGE_SORT(arr, depth)
IF length(arr) <= 1
   RETURN arr
Update maximum recursion depth
mid = length(arr)/2
left = MERGE_SORT(left half)
right = MERGE_SORT(right half)
Merge left and right
RETURN merged array
```

Code:

```
import time
import random
max_depth = 0
def merge_sort(arr, depth=1):
    global max_depth
    max_depth = max(max_depth, depth)
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left = merge_sort(arr[:mid], depth + 1)
    right = merge_sort(arr[mid:], depth + 1)
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] < right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result
sizes = [1000, 5000, 10000]
for size in sizes:
    arr = [random.randint(1, 100000) for _ in range(size)]
    max_depth = 0
    start = time.time()
```

```
    merge_sort(arr)
    end = time.time()
    print("\nInput Size:", size)
    print("Execution Time:", round(end - start, 6), "seconds")
    print("Maximum Recursion Depth:", max_depth)
```

Sample Output:

Input Size: 1000

Execution Time: 0.0032 seconds

Maximum Recursion Depth: 11

Input Size: 5000

Execution Time: 0.0165 seconds

Maximum Recursion Depth: 14

Input Size: 10000

Execution Time: 0.0351 seconds

Maximum Recursion Depth: 15

Analysis:

Stack Usage: Merge Sort creates recursive calls until the array size becomes one.

Recursion Depth: Depth ≈ $\log_2(n)$

For

$n = 1000 \rightarrow$ Depth ≈ 10

$n = 5000 \rightarrow$ Depth ≈ 13

$n = 10000 \rightarrow$ Depth ≈ 14

Thus stack growth is logarithmic.

Resource Utilization:

- CPU utilization increases due to recursive calls.
- Memory is consumed by:
    1. Recursive stack frames
    2. Temporary arrays during merging
- Systems with limited memory may experience overhead for very large inputs.

Optimization Techniques:

1. Iterative Merge Sort

Avoid recursion completely.

2. Bottom-Up Merge Sort

Merge subarrays iteratively.

3. Reduce Temporary Array Creation

Reuse auxiliary arrays.

4. Tail Recursion Optimization

Can reduce stack usage in languages that support it.

Time Complexity:

Best Case: O(n log n)

Average Case: O(n log n)

Worst Case: O(n log n)

Space Complexity: O(n)

Extra memory is required for mapping.

Result:

The experiment shows that Merge Sort's recursion depth grows logarithmically with input size. Although stack usage remains relatively small, recursive calls and temporary arrays consume additional memory. Iterative and bottom-up implementations can reduce stack overhead and improve memory efficiency on systems with limited resources.